

SOFTWARE REQUIREMENTS SPECIFICATION

AI Interview Preparation Platform

Version 1.0 | Phase-by-Phase Build Guide

Field	Details
Document Version	1.0
Project Name	AI Interview Preparation Platform
Prepared For	Full-Stack Developer (Interview Portfolio)
Tech Stack	React, Node.js, Express, PostgreSQL, OpenAI/Gemini API
Deployment	Vercel (Frontend) + Render (Backend) + Neon PostgreSQL
Timeline	4 Weeks (MVP)
Date	June 2025

Table of Contents

1. Introduction
2. Functional Requirements
3. Non-Functional Requirements
4. Database Schema
5. API Endpoints
6. Recommended Tech Stack
7. Folder Structure
8. 4-Week Development Roadmap
9. Stretch Goals
10. AI Feedback Prompt Template
11. Environment Variables
12. Requirement Summary

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines all functional and non-functional requirements for the AI Interview Preparation Platform. It is intended to guide development using GitHub Copilot inside VS Code. Each section maps directly to a build phase across 4 weeks.

1.2 Project Scope

The platform enables computer science students and job seekers to:

- Practice HR, DSA, OOP, DBMS, and OS interview questions
- Receive AI-generated feedback on their answers
- Track their DSA problem-solving progress
- Analyze strengths and weaknesses via a dashboard
- Get personalized question recommendations based on performance

1.3 Definitions & Acronyms

Term	Definition
FR	Functional Requirement
NFR	Non-Functional Requirement
JWT	JSON Web Token - used for authentication
DSA	Data Structures and Algorithms
OOP	Object-Oriented Programming
DBMS	Database Management System
OS	Operating Systems
API	Application Programming Interface
MVP	Minimum Viable Product
AI	Artificial Intelligence (OpenAI/Gemini API)

1.4 Document Overview

The document is organized into:

1. Section 2 - Functional Requirements (FR-1 to FR-12)
2. Section 3 - Non-Functional Requirements (NFR-1 to NFR-7)
3. Section 4 - Database Schema
4. Section 5 - API Endpoints
5. Section 6 - Tech Stack

- 6. Section 7 - Folder Structure
- 7. Section 8 - 4-Week Development Roadmap
- 8. Section 9 - Stretch Goals

2. Functional Requirements

The following requirements define what the system must do. Each FR is tagged with its module, priority, and the development week it belongs to.

2.1 User Management

FR-1: User Registration

Attribute	Detail
ID	FR-1
Module	User Management
Priority	High
Week	Week 1
Description	Users can create an account using email and password.
Endpoint	POST /register

Acceptance Criteria:

- User provides: name, email, password
- Email must be unique and validated via regex
- Password must be hashed using bcrypt before storage
- On success: return JWT token and user profile
- On failure: return descriptive error (email already exists, invalid format)

FR-2: User Login

Attribute	Detail
ID	FR-2
Module	User Management
Priority	High
Week	Week 1
Description	Authenticated users can log in with email and password.
Endpoint	POST /login

Acceptance Criteria:

- System validates credentials against stored bcrypt hash
- On success: return JWT token (expires in 7 days)
- On failure: return 401 Unauthorized with message
- Token must be stored in localStorage on client side

FR-3: User Profile

Attribute	Detail
ID	FR-3
Module	User Management
Priority	Medium
Week	Week 1
Description	Users can view and edit their profile, and see interview statistics summary.
Endpoint	GET /profile, PUT /profile

Acceptance Criteria:

- Displays: name, email, join date
- Shows quick stats: total interviews, average score, best category
- User can update: name
- Email change requires re-validation

2.2 Interview Module

FR-4: Create Interview Session

Attribute	Detail
ID	FR-4
Module	Interview
Priority	High
Week	Week 2
Description	Users select an interview category and start a new session.
Endpoint	POST /interview/start

Acceptance Criteria:

- User selects one of: HR, DSA, OOP, DBMS, OS
- System creates an interview record in the database
- Returns a session ID and the first question
- Session stores: user_id, type, start_time, status (active/completed)

FR-5: Question Delivery

Attribute	Detail
ID	FR-5
Module	Interview
Priority	High
Week	Week 2
Description	System presents questions one at a time based on selected category.
Endpoint	GET /questions?category=DSA&difficulty=medium

Acceptance Criteria:

- Questions are fetched from the questions table by category
- Presented one question at a time (no skipping ahead)
- Each question has: id, category, difficulty (easy/medium/hard), question_text
- System tracks which questions have been answered in the session

FR-6: Answer Submission

Attribute	Detail
ID	FR-6
Module	Interview
Priority	High
Week	Week 2
Description	Users submit text-based answers to interview questions.
Endpoint	POST /answer

Acceptance Criteria:

- User types their answer in a text area
- Answer is saved to the answers table with: interview_id, question_id, answer_text
- After submission, AI feedback is triggered (FR-7)
- Voice input is out of scope for v1.0 (Phase 2 feature)

FR-7: AI Feedback Generation

Attribute	Detail
ID	FR-7
Module	Interview
Priority	High
Week	Week 3

Attribute	Detail
Description	AI evaluates the submitted answer and returns structured feedback.
Endpoint	POST /feedback (internal service call)

Acceptance Criteria:

- System sends: question + user answer to OpenAI/Gemini API
- AI evaluates on three criteria: Relevance, Completeness, Communication

Response includes:

- Score: numeric (0-10)
- Strengths: list of positive aspects
- Improvements: list of specific suggestions
- Feedback stored in answers table: feedback JSON, score
- If AI API fails: retry once, then return generic error message

2.3 Analytics Module

FR-8: Analytics Dashboard

Attribute	Detail
ID	FR-8
Module	Analytics
Priority	High
Week	Week 3
Description	Dashboard displays aggregated performance statistics for the logged-in user.
Endpoint	GET /analytics

Acceptance Criteria:

- Displays: Total Interviews Taken
- Displays: Average Score (across all sessions)
- Displays: Best Topic (highest average score by category)
- Displays: Weakest Topic (lowest average score by category)
- Data loads within 2 seconds

FR-9: Progress Tracking

Attribute	Detail
ID	FR-9
Module	Analytics

Attribute	Detail
Priority	Medium
Week	Week 3
Description	Visual representation of score progress over time per category.
Endpoint	GET /progress

Acceptance Criteria:

- Shows progress chart for: HR, DSA, DBMS, OS, OOP
- Each chart shows scores over interview sessions (chronological)
- Uses a line chart or bar chart component on the frontend

2.4 DSA Tracker

FR-10: Problem Tracking

Attribute	Detail
ID	FR-10
Module	DSA Tracker
Priority	High
Week	Week 4
Description	Users log DSA problems they have solved or are attempting.
Endpoint	POST /problem, GET /problems

Acceptance Criteria:

- User can add a problem with: name, platform (LeetCode/GFG/etc), difficulty, topic, status
- Status options: Todo, In Progress, Solved
- Problems are stored in dsa_problems table per user
- Users can update status and delete problems

FR-11: Topic Analytics

Attribute	Detail
ID	FR-11
Module	DSA Tracker
Priority	Medium
Week	Week 4

Attribute	Detail
Description	Visual breakdown of solved problems by DSA topic.
Endpoint	GET /dsa/analytics

Acceptance Criteria:

- Shows completion percentage per topic: Arrays, Trees, Graphs, DP, Sorting, etc.
- Displayed as a progress bar or pie chart
- Highlights topics with < 50% completion as weak areas

2.5 Recommendation Engine

FR-12: Personalized Question Recommendations

Attribute	Detail
ID	FR-12
Module	Recommendations
Priority	Medium
Week	Week 4
Description	System suggests interview questions based on the user's weakest topics.
Endpoint	GET /recommendations

Acceptance Criteria:

- Identifies user's 2-3 lowest scoring categories from analytics
- Returns 5 recommended questions per weak category
- Example: if Graphs score < 50%, recommend BFS, DFS, Shortest Path questions
- Recommendations update after each completed interview session

3. Non-Functional Requirements

These requirements define system quality attributes. Interviewers often ask about these, so they must be addressed in both code and architecture decisions.

ID	Category	Requirement	Target / Implementation
NFR-1	Performance	API response time	< 500ms for all REST endpoints; < 2s for dashboard load
NFR-2	Scalability	User and session load	Support 1000+ registered users; 100+ concurrent sessions
NFR-3	Security	Auth and data protection	bcrypt password hashing; JWT authentication; input validation; rate limiting (100 req/min per IP)
NFR-4	Reliability	Fault tolerance	Database backups on Neon; Winston error logging; retry logic for failed AI API calls (1 retry)
NFR-5	Availability	Uptime target	99% uptime; deploy on Render with health check endpoint GET /health
NFR-6	Maintainability	Code architecture	Layered MVC architecture; modular APIs; service separation (authService, interviewService, aiService)
NFR-7	Usability	UI/UX standards	Responsive design with TailwindCSS; mobile-friendly layout; clean dashboard with clear loading/error states

3.1 Security Details (NFR-3)

- Passwords: hashed with bcrypt (salt rounds: 10)
- JWT: signed with SECRET_KEY from environment variable, expiry 7 days
- All protected routes: use authMiddleware to verify JWT
- Input validation: use express-validator on all POST body fields
- Rate limiting: use express-rate-limit middleware
- CORS: restrict origin to frontend domain in production

3.2 Error Handling Standards (NFR-4)

- All controllers wrapped in try-catch
- Centralized error handler middleware returns: { error: 'message', code: 400 }
- AI API failures return: { error: 'Feedback unavailable. Try again later.' }
- Database errors logged via Winston, never exposed to client

4. Database Schema

PostgreSQL (hosted on Neon). All tables use UUID or SERIAL for primary keys.

4.1 users

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-increment user ID
name	VARCHAR(100)	NOT NULL	User's full name
email	VARCHAR(255)	NOT NULL, UNIQUE	Login email
password_hash	VARCHAR(255)	NOT NULL	bcrypt hashed password
created_at	TIMESTAMP	DEFAULT NOW()	Account creation timestamp

4.2 interviews

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Interview session ID
user_id	INT	FK -> users.id	Owner of this session
type	VARCHAR(50)	NOT NULL	HR / DSA / OOP / DBMS / OS
score	DECIMAL(4,2)	NULLABLE	Aggregated session score (0-10)
status	VARCHAR(20)	DEFAULT 'active'	active / completed
created_at	TIMESTAMP	DEFAULT NOW()	Session start time

4.3 questions

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Question ID
category	VARCHAR(50)	NOT NULL	HR / DSA / OOP / DBMS / OS
difficulty	VARCHAR(20)	NOT NULL	easy / medium / hard
question_text	TEXT	NOT NULL	The interview question

4.4 answers

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Answer record ID

Column	Type	Constraints	Description
interview_id	INT	FK -> interviews.id	Parent session
question_id	INT	FK -> questions.id	Question answered
answer	TEXT	NOT NULL	User's text answer
feedback	JSONB	NULLABLE	AI feedback: { score, strengths, improvements }
score	DECIMAL(4,2)	NULLABLE	Per-answer AI score (0-10)

4.5 dsa_problems

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Problem record ID
user_id	INT	FK -> users.id	Owner
problem_name	VARCHAR(200)	NOT NULL	Problem title
platform	VARCHAR(100)	NULLABLE	LeetCode / GFG / HackerRank
topic	VARCHAR(100)	NOT NULL	Arrays / Graphs / DP etc.
difficulty	VARCHAR(20)	NOT NULL	easy / medium / hard
status	VARCHAR(20)	DEFAULT 'Todo'	Todo / In Progress / Solved

5. API Endpoints

5.1 Authentication

Method	Endpoint	Auth Required	Description
POST	/api/auth/register	No	Create new user account
POST	/api/auth/login	No	Login and receive JWT token
GET	/api/auth/profile	Yes (JWT)	Get current user profile
PUT	/api/auth/profile	Yes (JWT)	Update user name

5.2 Interviews

Method	Endpoint	Auth Required	Description
POST	/api/interview/start	Yes	Create new interview session
GET	/api/interview/questions	Yes	Get questions by category
POST	/api/interview/answer	Yes	Submit answer, trigger AI feedback
GET	/api/interview/results/:id	Yes	Get full interview results
GET	/api/interview/history	Yes	Get all past interviews

5.3 Analytics

Method	Endpoint	Auth Required	Description
GET	/api/analytics/dashboard	Yes	Get total interviews, avg score, best/worst topic
GET	/api/analytics/progress	Yes	Get score history per category
GET	/api/analytics/recommendations	Yes	Get personalized question recommendations

5.4 DSA Tracker

Method	Endpoint	Auth Required	Description
POST	/api/dsa/problem	Yes	Add new DSA problem
GET	/api/dsa/problems	Yes	Get all user's problems (filterable)
PUT	/api/dsa/problem/:id	Yes	Update problem status
DELET	/api/dsa/problem/:id	Yes	Delete a problem

Method	Endpoint	Auth Required	Description
E			
GET	/api/dsa/analytics	Yes	Get topic completion percentages

5.5 System

Method	Endpoint	Auth Required	Description
GET	/health	No	Health check for deployment monitoring

6. Recommended Tech Stack

Layer	Technology	Reason
Frontend	React + TailwindCSS	Component-based UI; utility-first CSS for rapid responsive design
Frontend Routing	React Router v6	SPA navigation between pages
HTTP Client	Axios	Promise-based HTTP with interceptors for JWT attachment
Backend	Node.js + Express	Non-blocking I/O; fast REST API development
Database	PostgreSQL (Neon)	Structured relations; great for analytics JOINS; common in interviews
Auth	JWT + bcrypt	Stateless auth; bcrypt for secure password hashing
AI	OpenAI API (or Gemini)	Feedback generation using GPT-4o or Gemini 1.5 Flash
Frontend Deploy	Vercel	Zero-config React deployment with CI/CD from GitHub
Backend Deploy	Render	Free tier Node.js hosting with auto-deploy from GitHub
DB Hosting	Neon PostgreSQL	Serverless PostgreSQL with free tier and backups
Logging	Winston	Structured logging for errors and AI failures
Validation	express-validator	Input sanitization on all API routes
Rate Limiting	express-rate-limit	Prevents API abuse (NFR-3)

7. Folder Structure

7.1 Backend

Path	Purpose
backend/src/app.js	Express app entry point; middleware setup; route imports
backend/src/config/db.js	PostgreSQL connection pool (pg library)
backend/src/config/env.js	Environment variable validation
backend/src/routes/auth.routes.js	Auth route definitions (register, login, profile)
backend/src/routes/interview.routes.js	Interview route definitions
backend/src/routes/analytics.routes.js	Analytics and recommendations routes
backend/src/routes/dsa.routes.js	DSA tracker routes
backend/src/controllers/auth.controller.js	Business logic for auth endpoints
backend/src/controllers/ interview.controller.js	Business logic for interview session
backend/src/controllers/ analytics.controller.js	Business logic for dashboard data
backend/src/controllers/dsa.controller.js	Business logic for DSA problem CRUD
backend/src/services/ai.service.js	OpenAI/Gemini API calls; prompt engineering
backend/src/services/interview.service.js	Interview session helpers; score aggregation
backend/src/middleware/auth.middleware.js	JWT verification middleware
backend/src/middleware/error.middleware.js	Centralized error handler
backend/src/models/	SQL query helpers per table (users, interviews, answers, dsa_problems)
backend/src/utils/logger.js	Winston logger configuration
backend/.env	API keys, DB URL, JWT secret (never commit to git)

7.2 Frontend

Path	Purpose
frontend/src/App.jsx	Root component; React Router setup
frontend/src/pages/LoginPage.jsx	Login form and logic
frontend/src/pages/RegisterPage.jsx	Registration form
frontend/src/pages/DashboardPage.jsx	Analytics overview and stats

Path	Purpose
frontend/src/pages/InterviewPage.jsx	Active interview session UI
frontend/src/pages/ResultsPage.jsx	Interview results and AI feedback display
frontend/src/pages/DSATrackerPage.jsx	DSA problem tracker CRUD UI
frontend/src/pages/HistoryPage.jsx	Past interview history list
frontend/src/components/Navbar.jsx	Top navigation with auth state
frontend/src/components/QuestionCard.jsx	Single question + answer textarea
frontend/src/components/FeedbackCard.jsx	AI feedback display: score, strengths, improvements
frontend/src/components/ProgressChart.jsx	Line/bar chart for score history
frontend/src/components/ DSAProblemForm.jsx	Form to add/edit DSA problems
frontend/src/hooks/useAuth.js	Custom hook for auth state
frontend/src/services/api.js	Axios instance with base URL and JWT interceptor
frontend/src/context/AuthContext.jsx	Global auth state (user, token, logout)

8. 4-Week Development Roadmap

Week 1 - Foundation (Auth + Project Setup)

Goal: User can register, login, and see an empty dashboard skeleton.

Task	Layer	FR/NFR Covered
Initialize Node.js + Express project	Backend	-
Setup PostgreSQL on Neon, create users table	Database	FR-1, FR-2
Build POST /register with bcrypt hashing	Backend	FR-1, NFR-3
Build POST /login with JWT generation	Backend	FR-2, NFR-3
Build authMiddleware (JWT verification)	Backend	NFR-3
Build GET /profile endpoint	Backend	FR-3
Setup React app with TailwindCSS	Frontend	NFR-7
Build Login page + Register page	Frontend	FR-1, FR-2
Build AuthContext + useAuth hook	Frontend	FR-2
Build Navbar with auth-aware links	Frontend	FR-3
Build Dashboard skeleton (empty state)	Frontend	FR-8
Setup Axios instance with JWT interceptor	Frontend	NFR-3

Week 2 - Interview Engine

Goal: User can complete a full interview session (no AI yet).

Task	Layer	FR/NFR Covered
Create interviews, questions, answers tables	Database	FR-4, FR-5, FR-6
Seed questions table (50+ questions per category)	Database	FR-5
Build POST /interview/start	Backend	FR-4
Build GET /interview/questions (by category)	Backend	FR-5

Task	Layer	FR/NFR Covered
Build POST /interview/answer (save answer)	Backend	FR-6
Build GET /interview/results/:id	Backend	FR-7
Build GET /interview/history	Backend	FR-9
Build Interview page (category selector)	Frontend	FR-4
Build QuestionCard component	Frontend	FR-5
Build answer submission flow	Frontend	FR-6
Build ResultsPage (placeholder feedback)	Frontend	FR-7

Week 3 - AI Integration + Analytics

Goal: AI evaluates answers and dashboard shows real data.

Task	Layer	FR/NFR Covered
Setup OpenAI/Gemini API key in .env	Backend	FR-7
Build ai.service.js with prompt template	Backend	FR-7
Integrate AI call on POST /interview/answer	Backend	FR-7, NFR-4
Handle AI errors with retry logic	Backend	NFR-4
Build GET /analytics/dashboard query (JOINS)	Backend	FR-8
Build GET /analytics/progress query	Backend	FR-9
Build GET /analytics/recommendations logic	Backend	FR-12
Update FeedbackCard with real AI response	Frontend	FR-7
Build ProgressChart with Recharts library	Frontend	FR-9
Populate Dashboard with real API data	Frontend	FR-8
Add loading states and error handling to all pages	Frontend	NFR-7

Week 4 - DSA Tracker + Deployment + Polish

Goal: Production-ready MVP live on Vercel + Render.

Task	Layer	FR/NFR Covered
Create dsa_problems table	Database	FR-10
Build DSA CRUD endpoints (POST, GET, PUT, DELETE)	Backend	FR-10
Build GET /dsa/analytics (topic breakdown)	Backend	FR-11
Build DSATrackerPage with problem table + filters	Frontend	FR-10
Build topic analytics chart (progress bars)	Frontend	FR-11
Add GET /health endpoint	Backend	NFR-5
Configure CORS for production origin	Backend	NFR-3
Deploy backend to Render with env vars	DevOps	NFR-5
Deploy frontend to Vercel with REACT_APP_API_URL	DevOps	NFR-5
Add Winston logging throughout backend	Backend	NFR-4
Mobile responsiveness pass on all pages	Frontend	NFR-7
Final testing: all FR scenarios end-to-end	QA	All FRs

9. Stretch Goals (Phase 2 - Post-MVP)

Only implement these if all Week 1-4 goals are completed. These make excellent talking points in interviews.

Feature	Technology	Complexity	Value
Voice Interviews	OpenAI Whisper API	High	Simulates real interview environment
Real-Time Interview (Timed)	Socket.IO	High	Adds pressure simulation
Resume-Based Questions	PDF parse + AI prompt	Medium	Personalizes experience
Coding Round Simulator	Monaco Editor	High	Rounds out technical interview prep
Email Reminders	SendGrid / Nodemailer	Low	Increases user retention
Interview Sharing	Unique share links	Medium	Virality feature

10. AI Feedback Prompt Template

Use this exact prompt structure in `ai.service.js` for consistent feedback from OpenAI/Gemini.

System Prompt:

You are an expert technical interviewer. Evaluate the candidate's answer and respond ONLY in this JSON format:

```
{ "score": 0-10, "strengths": ["...", "..."], "improvements": ["...", "..."] }
```

User Prompt:

Question: {question_text}

Candidate Answer: {user_answer}

Category: {category} | Difficulty: {difficulty}

Scoring Rubric (for reference in code comments):

- 9-10: Comprehensive, accurate, well-communicated
- 7-8: Mostly correct, minor gaps
- 5-6: Partially correct, key concepts missing
- 3-4: Significant gaps or misconceptions
- 0-2: Off-topic or incorrect

11. Environment Variables

Create a `.env` file in `backend/` and add to `.gitignore` immediately.

Variable	Example Value	Used In
PORT	5000	app.js server listen
DATABASE_URL	postgresql:// user:pass@neon.tech/dbname	config/db.js
JWT_SECRET	your_super_secret_key_here	auth.service.js
JWT_EXPIRY	7d	auth.service.js
OPENAI_API_KEY	sk-...	ai.service.js
GEMINI_API_KEY	Alza...	ai.service.js (alternative)
NODE_ENV	development / production	app.js, CORS config
FRONTEND_URL	https://your-app.vercel.app	CORS origin in production

12. Requirement Summary

ID	Requirement	Priority	Week
FR-1	User Registration	High	1
FR-2	User Login (JWT)	High	1
FR-3	User Profile	Medium	1
FR-4	Create Interview Session	High	2
FR-5	Question Delivery	High	2
FR-6	Answer Submission	High	2
FR-7	AI Feedback Generation	High	3
FR-8	Analytics Dashboard	High	3
FR-9	Progress Tracking	Medium	3
FR-10	DSA Problem Tracking	High	4
FR-11	DSA Topic Analytics	Medium	4
FR-12	Personalized Recommendations	Medium	4
NFR-1	Performance < 500ms	High	All
NFR-2	Scalability 1000+ users	Medium	All
NFR-3	Security (bcrypt + JWT + rate limit)	High	1
NFR-4	Reliability (logging + retry)	High	3-4
NFR-5	Availability 99% uptime	Medium	4
NFR-6	Maintainability (layered arch)	High	All
NFR-7	Usability (responsive UI)	High	All

Total: 12 Functional Requirements + 7 Non-Functional Requirements = 19 Requirements covered in MVP v1.0.

— End of SRS Document —